SIMPLY FPU

by **Raymond Filiatreault**

Chap. 5
Data transfer instructions - Integer numbers

The FPU instructions covered in this chapter perform no mathematical operation on numerical data. Their main purpose is simply to transfer integer data between the FPU's 80-bit data registers and memory. *(Other instructions must be used to transfer floating point and BCD data).*

Integer data cannot be transferred directly between the CPU and FPU registers.

The integer data transfer instructions are:

FILD Integer LoaD from memory

FIST Integer STore to memory

FISTP Integer STore to memory and Pop the top data register

FISTTP Integer STore to memory after Truncation and Pop the top data register

Note that all FPU instructions using Integer data as an operand have an "I" immediately following the "F" in the mnemonic to distinguish them from instructions using other data types.

FILD (Load integer number from memory)

Syntax: **fild *Src***

Exception flags: Stack Fault, Invalid operation

This instruction decrements the TOP register pointer in the Status Word and loads the integer value from the specified source (*Src*) in the new TOP data register ST(0) after converting it to the 80-bit REAL10 format. The source must be the memory address of a signed 16-bit WORD integer, a signed 32-bit DWORD integer, or a signed 64-bit QWORD integer.

If the ST(7) data register which would become the new ST(0) is not empty, both a **S**tack **F**ault and an **I**nvalid operation exceptions are detected, setting both flags in the Status Word. The TOP register pointer in the Status Word would still be decremented and the new value in ST(0) would be the INDEFINITE NAN.

The content of the Tag Word for the new TOP register will be modified according to the result of the operation.

Examples of using this instruction:

```
fild word ptr[ebx+8]  ;loads the 5th WORD of an integer array addressed by EBX

fild int_var          ;loads the value of the integer variable int_var
                      ;according to its declared size

fild dword ptr[ebp+12]   ;typical assembler coding to load the value of a
                     ;dword integer passed on the stack as a parameter to a
                     ;procedure (the source file may have referred to it
                     ;using the name of a variable)
```

(See Chap.2 for integer data types, their addressing modes, and examples of using integer values from CPU registers.)

FIST (Store an integer number to memory)

```
Syntax:    fist Dest

Exception flags: Stack Fault, Invalid operation, Precision
```

This instruction rounds the value of the TOP data register ST(0) to the nearest integer according to the rounding mode of the RC field in the Control Word, and stores it at the specified destination (*Dest*). The destination can be the memory address of a 16-bit WORD or of a 32-bit DWORD.

> *This instruction cannot store the value of ST(0) as a 64-bit QWORD integer; see the FISTP instruction for such an operation.*

If the ST(0) data register is empty, both a **S**tack **F**ault and an **I**nvalid operation exceptions are detected, setting both flags in the Status Word. The value of 8000h would be stored at the specified destination if a WORD type, or 80000000h if a DWORD type.

If the rounded value of ST(0) would be larger than acceptable for the integer type of the destination, an **I**nvalid operation exception is detected, setting the related flag in the Status Word. The value of 8000h would be stored at the specified destination if a WORD type, or 80000000h if a DWORD type.

> Note: Although these 8000h and 80000000h integer values are stored as "error" codes, the FPU will treat them as -2^{15} and -2^{31} respectively if loaded back in a data register or used in comparison and arithmetic instructions.

If the value in ST(0) would contain a binary fraction, some precision would be lost and a **P**recision exception would be detected, setting the related flag in the Status Word.

The actual value in ST(0) does not change with this instruction.

Examples of using this instruction:

```
fist  word ptr [edi+ebx] ;stores the rounded value of ST(0) as a WORD
                         ;at the memory address indicated by EDI+EBX

fist  integer_var        ;stores the rounded value of ST(0)
                         ;at the memory variable integer_var
                         ;according to its declared integer data type.
```

> (See Chap.2 for integer data types, their addressing modes, and examples of transferring integer values to CPU registers.)

FISTP (Store an integer number to memory and Pop the top data register)

```
Syntax:    fistp Dest

Exception flags: Stack Fault, Invalid operation, Precision
```

This instruction is the same as the FIST instruction except for the following two additions:

- it allows storing the ST(0) value to memory as a 64-bit QWORD integer, and
- the ST(0) register is POPped after the transfer is completed, modifying the Tag Word and incrementing the TOP register pointer of the Status Word.

If an invalid exception is detected when attempting to store a QWORD, a value of 8000000000000000h would be stored at the specified destination.

This instruction would be used when the final result of a computation must be stored in memory as an integer, liberating the FPU's TOP data register for future use.

```
fistp  int_var  ;stores the rounded value of ST(0)
                ;at the memory variable int_var
                ;according to its declared integer data type
                ;and POPs the FPU's TOP data register
```

(See Chap.2 for integer data types, their addressing modes, and examples of transferring integer values to CPU registers.)

FISTTP (Store a truncated integer number to memory and Pop the top data register)

```
Syntax:    fisttp Dest

Exception flags: Stack Fault, Invalid operation, Precision
```

Note: This FPU instruction was introduced along with the SSE3 instruction set (although it is not an SSE3 instruction). It must not be used unless the computer is also capable of executing SSE3 instructions. It may not be supported by some assemblers and suggested encodings are provided to facilitate hard-coding of this instruction if not supported.

This instruction is the same as the FISTP instruction except for the following:

- it truncates the value of the TOP data register ST(0) to the nearest integer regardless of the rounding mode of the RC field in the Control Word.

This instruction would be used when the final result of a computation must be stored in memory as a truncated integer (without the need to modify the rounding mode of the Control Word), and then liberate the FPU's TOP data register for future use.

```
fisttp  int_var ;stores the truncated value of ST(0)
                ;at the memory variable int_var
                ;according to its declared integer data type
                ;and POPs the FPU's TOP data register
```

(See Chap.2 for integer data types, their addressing modes, and examples of transferring integer values to CPU registers.)

Being a recently added instruction on Intel chips along with the SSE3 instruction set, many assemblers may not be able to code it. Because the instruction requires a pointer to a memory address, and hand-coding all the possible variations of addressing modes could be very cumbersome, the following opcodes assume that the EAX register contains the pointer.

```
Encoding      Description
 DF 08     Store as a truncated WORD
 DB 08     Store as a truncated DWORD
 DD 08     Store as a truncated QWORD
```

Examples of using the hand-coding to store the value of ST(0) as a truncated DWORD could be as follows, assuming that *int_var* would be used as a DWORD integer variable (or [edi+ecx*4] is pointing into an integer DWORD array):

```
   lea   eax,int_var
   db    0DBh,8

or

   lea   eax,[edi+ecx*4]
   dw    8DBh
```

RETURN TO
SIMPLY FPU
CONTENTS